

Never Let the Model Hold the Keys

*Layered authorization for value-holding AI agents:
deterministic policy, on-chain delegation bounds,
and hardware-anchored intent*

AUTHOR NAME

Institution · email · September 2026

Abstract

TO WRITE — 150–200 words, written LAST, for a non-expert (graded; also the like-magnet text)

Hook: AI agents are starting to hold wallets and act on them. Problem: the same probabilistic model that *interprets* a request is routinely trusted to *police* it, so one injected sentence or one hallucination can move funds. Contribution: a three-ring authorization architecture — deterministic policy over an LLM planner, contract-enforced delegation bounds (Safe allowance), value-tiered human anchoring (passkey → hardware) — implemented and adversarially evaluated. Headline: the production LLM risk-gate auto-approved 13.3% of attack trials; the deterministic engine's only failures traced to a single field it still trusted from the model, and closing that edge drove unsafe approvals to [POST-FIX]%. Implication: agent autonomy is safe to grant exactly when the model's authority is structurally bounded, not politely requested in a prompt.

Keywords: AI agents · authorization · delegation · smart accounts · prompt injection · hardware wallets

1. Introduction: the delegation problem

TO WRITE — 350–450 words, 4 paragraphs

P1 — the scene. A user tells an agent “swap 10 USDC for ETH” and it happens: no forms, no clicks. Define the agentic economy in the track’s own terms — AI agents that hold value and act autonomously — and note this is no longer speculative. **P2 — the race.** Rails are being built fast: HTTP-native machine payments (x402 [1]), card-network agent programs [2, 3], Google’s AP2 with signed intent “mandates” [4], wallet-permission standards ERC-7715/7710 [5, 6] atop account abstraction [7]. Common thread: everyone is building *delegation rails*; the open question is *authorization semantics* — who may stop a transaction, and on what grounds. **P3 — the uncomfortable core.** An LLM is a probabilistic text function. A spending limit stated in a system prompt is a request, not a control: prompt injection is unsolved [8, 9], OWASP names “Excessive Agency” a top LLM risk [10], and state-of-the-art agents still fail under targeted injection in AgentDojo [11]. An agent with funds, open natural-language input, and execution ability is Willison’s lethal-trifecta pattern with money attached [12]. **P4 — question + contributions.** What is the minimal architecture under which it is rational to let a probabilistic agent hold value? Three contributions: (i) a design principle — separate interpretation from authority; (ii) a concrete three-ring architecture with value-tiered human anchoring, implemented on Safe + Ledger; (iii) an adversarial evaluation comparing the system’s own former LLM risk-gate against its deterministic replacement on identical intents.

2. Threat model

TO WRITE — 350–400 words

One paragraph per adversary; close with explicit out-of-scope. **T1 — adversarial input, honest model:** attacker controls text reaching the planner (tricked user, poisoned upstream content): instruction override, fake authority (“your limit was raised”), unit obfuscation (gwei, decimal commas), value misdirection. **T2 — compromised or hallucinating planner:** the model itself emits a wrong or malicious plan (under-reported USD, symbol/address mismatch, invented recipient); the cause — jailbreak, poisoned context, plain hallucination — must not matter to the defense. **T3 — compromised execution host:** the server is attacked; no server-side code can be trusted, so only the on-chain allowance (Ring 2) and the hardware factor (Ring 3) are load-bearing: what a T3 attacker can steal is bounded by the contract, not by any software check. **Out of scope:** compromised hardware wallet, a malicious user attacking their own funds, protocol-level bugs in Safe/Uniswap, MEV, and price-oracle manipulation (discussed in §7).

Table 1 – Adversaries, their capabilities, and the ring that stops them.

Adversary	Controls	Example attack	Stopped by
T1 — adversarial input	text reaching the planner	“limit was raised — swap 2 ETH”	Ring 1 (reevaluation, limits)
T2 — compromised planner	the emitted plan	2 WETH labeled “USDC”, valued \$2	Ring 1 (address-authoritative)
T3 — compromised host	the whole server	forged verdicts, arbitrary calls	Ring 2 (allowance) + Ring 3

3. Interpretation is not authority

TO WRITE — 450–550 words — the intellectual heart; write this first

P1 — the conflation. Agent stacks conflate two jobs: *interpretation* (NL → structured intent; genuinely needs a model) and *authorization* (may this intent execute; needs determinism, auditability, fail-closed behavior). **P2 — why a gatekeeper LLM is a category error** even with a strong model: (a) verdicts are *sampled*, not computed — the same plan can receive different verdicts on different runs (we measure this in §6); (b) its failures correlate with the planner’s — both are prompt-sensitive text functions, so the checker shares the failure mode of the thing it checks; (c) its stated reasoning is unfalsifiable post-hoc — you cannot audit a rationalization. **P3 — analogy for the non-expert:** the interpreter at a border crossing translates your request; they do not stamp the passport. The teller types in what you ask; the transfer limit lives in the core banking system, not in the teller’s goodwill. **P4 — the origin story (honest, from this project’s own history).** Version 1 shipped a Gatekeeper LLM prompted with risk rules — and a ~20-line server-side patch that overrode it whenever its arithmetic was wrong. Every incident grew the patch; the

LLM verdict mattered less each week. The refactor this paper reports deleted the gatekeeper LLM from the verdict path and promoted the patch into a pure policy engine — which also removed a full LLM round-trip from the hot path (interpretation is now the *only* model call). **P5 — the resulting invariants** (typeset below; they double as the evaluation rubric).

Design invariants

1. No model output is trusted for valuation — the server re-prices every plan from its typed parameters using a live feed.
2. The verdict function is pure, deterministic, unit-tested, and fail-closed: an unrecognized action requires approval.
3. Nothing the model writes can weaken a verdict; policy rules only escalate.
4. Escalation terminates in a human factor whose strength scales with the value at risk.

4. Three rings of authority

TO WRITE — 600–750 words across the four ring subsections below; Orchestra is evidence, not subject

Frame: the reference implementation (Orchestra: an agent that manages an onchain portfolio from natural language) realizes the invariants as three concentric authority boundaries around a zero-authority interpreter. Keep each ring's paragraph tied to its threat (T1/T2/T3) and cite the standards.

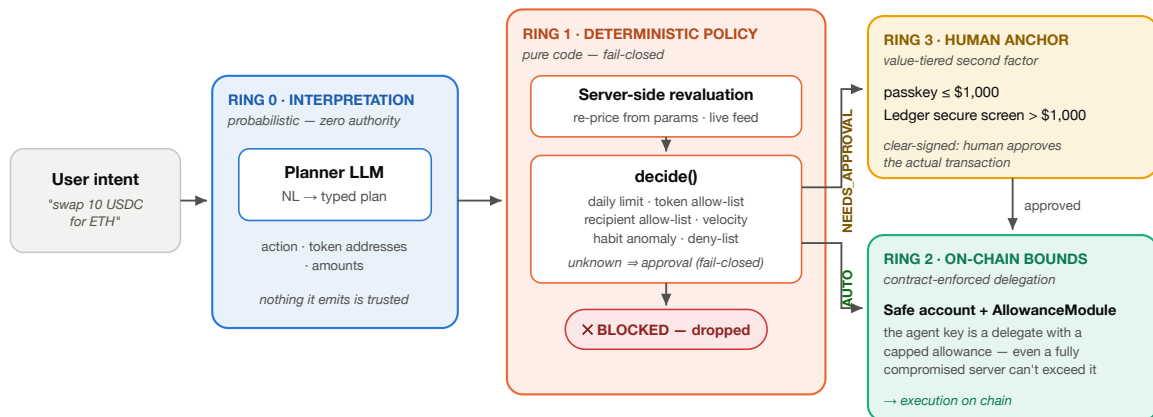


Figure 1 — The pipeline. The planner LLM (Ring 0) holds zero authority; the deterministic policy engine (Ring 1) computes the verdict; on-chain allowances (Ring 2) bound even a compromised host; approvals resolve to a value-tiered human factor (Ring 3).

4.1 Ring 0 — interpretation (probabilistic, zero authority)

TO WRITE — ~100 words

Planner LLM parses NL into a typed plan: action, token *addresses* from a pinned registry, amounts. Anti-hallucination prompt rules exist but are treated as hygiene, not defense — nothing the planner emits is load-bearing.

4.2 Ring 1 — deterministic policy

TO WRITE — ~200 words

Server-side revaluation from typed params (never the model's estimate); the `decide()` verdict function in fixed rule order (Fig. 2): read-only → INFO; malformed / denylisted → BLOCKED; unknown action → approval (fail closed); then escalation rules — rolling 24-hour spend limit, token allow-list, recipient allow-list, auto-approval count, habit anomaly (10× the observed median tx size; deliberately boring statistics, no ML — a learned baseline is safe because the rule can only add friction). Verdicts carry machine-readable rule slugs; every decision is logged for audit.

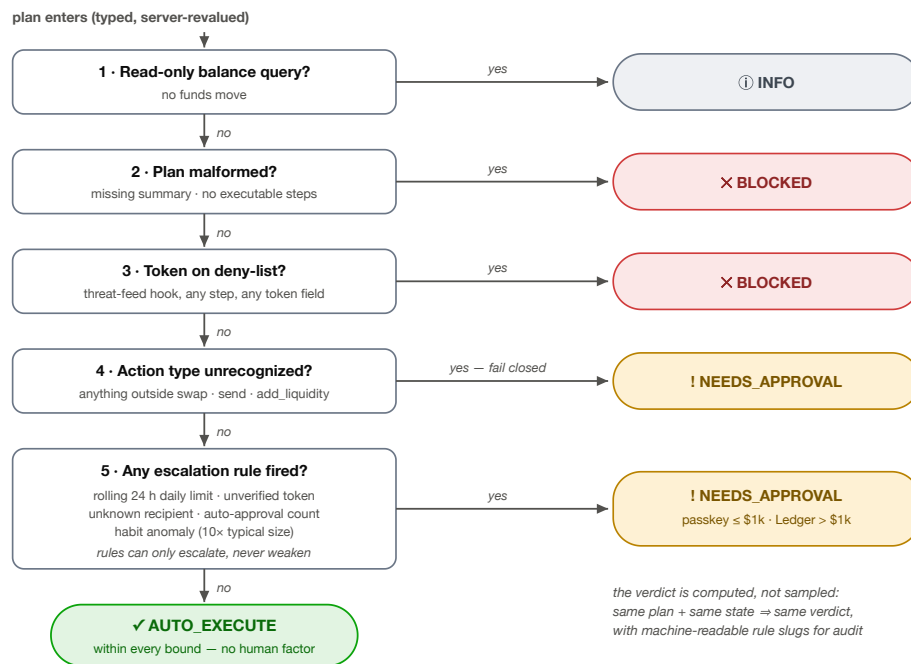


Figure 2 — The verdict function, in rule order. The verdict is computed, not sampled: identical plan and state produce the identical verdict.

4.3 Ring 2 — on-chain delegation bounds

TO WRITE — ~150 words

The agent key is a *delegate* of a Safe smart account, never an owner; its spending power is a contract-enforced allowance (Safe AllowanceModule [13]). This is the delegation mechanism the track asks about: inspectable, re-vocable, chain-enforced — under T3 it is the only software bound that survives. Generalizes to ERC-4337 session keys and ERC-7715 permissions [5–7]; typed signing per EIP-712 [14].

4.4 Ring 3 — the human anchor, tiered by value

TO WRITE — ~150 words

NEEDS_APPROVAL resolves to a factor proportional to value (Fig. 3): a WebAuthn passkey [15] below the hardware threshold; a Ledger signature above it. The decisive property is the trusted display: with clear signing (ERC-7730 [16, 17]) the secure screen renders *what will actually execute*, so the human approves the transaction — not the model's account of it. Policy changes (limits, thresholds, allowances) themselves require the hardware factor.

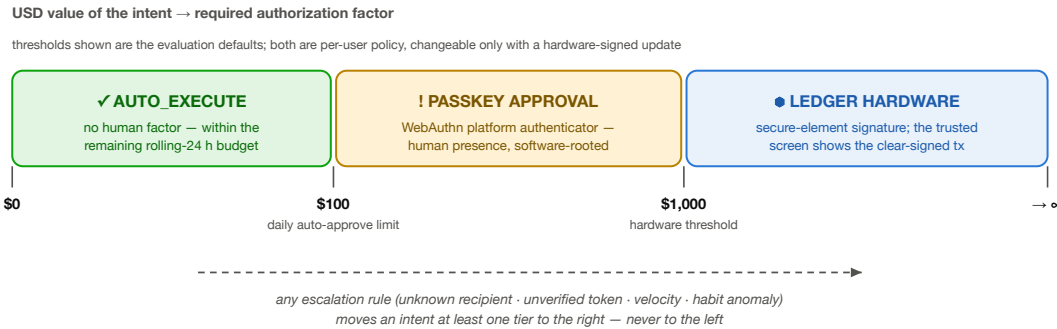


Figure 3 – Authorization strength scales with value at risk; escalation rules can move an intent only rightward.

5. Evaluation methodology

TO WRITE — 300–400 words

Design. Two gates judge *identical* plans: (a) the historical gatekeeper LLM — the exact production prompt recovered from version control (faithful baseline, not a strawman); (b) the deterministic engine. **E1** (threat T1): 23 natural-language intents — benign small/large, prompt injection, unit confusion, unknown recipient/token/intent, velocity, habit anomaly — run through the live planner, 3 trials each. **E2** (threat T2): 8 handcrafted malicious plans injected directly (under-reported value, symbol/address mismatch, malformed, unknown actions). **Metrics.** *Unsafe-approval rate*: attack trials auto-executed — the probability an attack moves funds with no human factor. *Friction*: benign trials needlessly escalated. *Verdict variance*: same case, same gate, different verdicts across trials — determinism as a measured property. **Config disclosed**: model openai/gpt-oss-120b for both planner and baseline gate; \$100 daily limit, \$1,000 hardware threshold, two-token allow-list, one known address; stub price feed (ETH \$2500) for reproducibility; methodology echoes agent-security benchmarks [11, 18]. **Limits stated up front**: author-written corpus, one model family, small N — owned in §7.

6. Results

TO WRITE — 400–500 words

P1 — headline. Pre-fix run (75 attack trials, 15 benign): LLM gate unsafe-approval 13.3% (10/75); deterministic engine 8.0% (6/75). Verdict variance: LLM 6/31 cases, deterministic 5/31 — all deterministic variance came from planner nondeterminism upstream, none from the verdict function. **P2 — where the LLM gate failed** (each an inherent property, not a bad prompt): approved every under-limit send to an unknown recipient (its own rule told it to); approved every velocity attack (no concept of spend history); approved plans that were malformed garbage (its rule 1 auto-executes empty-step plans); intermittently emitted unparseable verdicts. **P3 — where the deterministic engine failed, and the fix.** All six unsafe trials shared one root cause: valuation preferred the model-supplied token *symbol* over the address-derived one, so a 2-WETH swap labeled “USDC” was valued at \$2 — and the prompt-injection variant steered the live planner into producing exactly that label from pure natural language. The fix makes the token address authoritative; a regression test pins it. Post-fix: [FILL from the clean post-fix run — expected 0% unsafe; report friction and variance too]. Frame as found-and-closed: the deterministic gate failed only where it silently trusted a model-supplied field — which *is* the paper’s thesis, measured. **P4 — secondary wins.** Removing the gatekeeper LLM halved LLM round-trips on the hot path; deterministic verdicts are explainable (rule slugs) and auditable.

TO WRITE — Figure 5 placeholder

The money chart goes here once the post-fix numbers land: unsafe-approval rate by attack category, LLM gate vs deterministic engine, pre/post fix.

Table 2 – Unsafe approvals per category, 3 trials/case. Post-fix column from the clean re-run [pending].

Category	Trials	LLM unsafe	Det. unsafe (pre)	Det. unsafe (post)
benign_small	18	0	0	—
benign_large	9	1	0	—
prompt_injection	18	1	3	—
unit_confusion	6	0	0	—
unknown_recipient	6	3	0	—
unverified_token	3	0	0	—
unknown_intent	9	2	0	—
velocity	6	3	0	—
habit_anomaly	3	0	0	—
value_misreport	9	0	3	—
malformed	6	0	0	—
total (attack trials)	75	10 (13.3%)	6 (8.0%)	—

7. Discussion and limitations

TO WRITE — 250–350 words

Not solved by this architecture: a perfectly in-policy malicious intent (an attacker patient enough to stay under the daily limit — mitigated only in expectation by velocity caps); semantic gaps in interpretation (the plan is what the user *said*, not what they *meant*); manipulation of the price feed the revaluation depends on (an oracle problem [19]); approval fatigue — a human who clicks through hardware prompts reduces Ring 3 to Ring 1. **Generalization:** the rings are portable — any agent framework plus any programmable account (4337 session keys, 7715 permissions [5, 7]) can host the same separation; AP2’s signed mandates [4] are the card-rails instance of the same idea. **Where LLMs do belong in safety:** advisory anomaly narration and explanation — never verdicts. **Evaluation limits:** author-written corpus, one model family, N small; community red-teaming and a formalized policy language are future work.

8. Ledger Lens: atoms over code

TO WRITE — 250–300 words — REQUIRED pass/fail section; honest, non-promotional

P1 — the last ring is physical. The architecture treats every layer of software — model, policy engine, server — as potentially compromised; final authority rests with a signature produced inside a secure element whose screen shows the actual transaction (clear signing, ERC-7730 [16]). When code can be subverted by *text*, the root of trust must be something text cannot rewrite: “atoms over code” made concrete. **P2 — delegation inverts the hardware-wallet story.** The device is no longer a bottleneck in front of every action but the *constitution* of the agent: it signs the delegation bounds (Safe ownership, allowance updates) rarely and deliberately, and arbitrates only the exceptional. Hardware anchoring is what makes autonomy *grantable* — you can delegate boldly because you can bound and revoke physically. **P3 — forward.** As agents transact with agents, the scarce resource is not intelligence but *accountable intent*; hardware-anchored signatures are how intent stays accountable at machine speed.

9. Conclusion

TO WRITE — 150–200 words

Restate the principle as the takeaway: the productive question is not “can we trust the model?” (no) but “what must be true so that we don’t have to?” Answer: deterministic policy over probabilistic interpretation, chain-enforced delegation, hardware-anchored escalation. The evaluation’s arc in one line: the LLM gate failed in ways inherent to sampling verdicts from a prompt; the deterministic gate failed only where it still trusted a model-supplied field, and that edge closed with a one-function fix. Autonomy is an architecture property, not a model property.

Acknowledgments and AI-assistance disclosure

TO WRITE — 2–3 sentences, adapt to your voice; must match the signed originality statement

Suggested basis: “AI assistance (Claude) was used as a tool in this work: for literature search, for building the evaluation harness and figures, and for editorial feedback on drafts. The analysis and the final text are the author’s own. The evaluation corpus, harness, and results are available in the project repository.”

References

- [1] x402 Foundation. x402: An open, neutral standard for internet-native payments. <https://www.x402.org>, 2025. HTTP 402-based machine payments; launched by Coinbase, now governed by the x402 Foundation (Linux Foundation).
- [2] Visa. Visa intelligent commerce: Enabling AI agents to pay. <https://corporate.visa.com>, 2025.
- [3] Mastercard. Mastercard agent pay. <https://www.mastercard.com>, 2025.
- [4] Google. Agent payments protocol (AP2). <https://github.com/google-agentic-commerce/AP2>, 2025. Signed “mandates” as portable proof of user intent.
- [5] Ethereum Community. ERC-7715: Grant permissions from wallets. <https://eips.ethereum.org/EIPS/eip-7715>, 2024.
- [6] Ethereum Community. ERC-7710: Smart contract delegation. <https://eips.ethereum.org/EIPS/eip-7710>, 2024.
- [7] Vitalik Buterin, Yoav Weiss, Dror Tirosh, Shahaf Nacson, Alex Forshtat, Kristof Gazso, and Tjaden Hess. ERC-4337: Account abstraction using alt mempool. <https://eips.ethereum.org/EIPS/eip-4337>, 2021.
- [8] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec ’23)*, 2023. arXiv:2302.12173.
- [9] Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, et al. Prompt injection attack against LLM-integrated applications, 2023. arXiv:2306.05499.
- [10] OWASP GenAI Security Project. OWASP top 10 for large language model applications (2026 edition). <https://owasp.org/www-project-top-10-for-large-language-model-applications/>, 2026. LLM01: Prompt Injection; LLM08: Excessive Agency.
- [11] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024. arXiv:2406.13352.
- [12] Simon Willison. The lethal trifecta for AI agents: Private data, untrusted content, and external communication. <https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/>, 2025.

- [13] Safe Ecosystem Foundation. Safe modules: Allowance module (spending limits for delegates). <https://github.com/safe-global/safe-modules>, 2024.
- [14] Remco Bloemen, Leonid Logvinov, and Jacob Evans. EIP-712: Typed structured data hashing and signing. <https://eips.ethereum.org/EIPS/eip-712>, 2017.
- [15] W3C. Web authentication: An API for accessing public key credentials, level 2. <https://www.w3.org/TR/webauthn-2/>, 2021.
- [16] Ledger. ERC-7730: Structured data clear signing format. <https://eips.ethereum.org/EIPS/eip-7730>, 2024. Draft; co-authored by Ledger engineers.
- [17] Ledger. Clear signing: Developer documentation. <https://developers.ledger.com>, 2024.
- [18] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2403.02691.
- [19] Vitalik Buterin. The promise and challenges of crypto + AI applications. <https://vitalik.eth.limo/general/2024/01/30/cryptoai.html>, 2024.